가이드 매뉴얼 · 강의덱

Claude Code 팀 운영법

폴더 하나에 세션 하나, 역할을 나눠 품질을 지키는 구조

폴더와 마크다운 파일, 세션 간 메시지만으로 만든다

역할마다 폴더를 나누고, 설계 승인을 받은 뒤 구현하고, 검수를 두 번 거친다.
위험한 명령은 도구로 막고, 배운 것은 파일로 남기는 팀을 처음부터 따라 만든다.

세 가지 원칙이 팀 전체를 움직인다

프레임워크나 별도 서비스 없이 폴더·마크다운·메시지만 쓴다

1

세션 하나는 책상 하나다

역할마다 폴더를 두고 그 폴더에서 Claude Code를 실행한다. 대화가 섞이지 않고, 누가 무엇을 하는지 폴더만 봐도 보인다.

2

설계 승인 전에는 코드를 쓰지 않는다

과제 의도를 INTENT.md에 적고 설계안이 승인된 뒤에만 구현한다. 엉뚱한 방향으로 토큰을 쓰는 일이 줄어든다.

3

배운 것은 파일로 남긴다

세션은 끝나면 대화를 잊는다. 지적은 MEMORY.md에, 반복된 지적은 HANDBOOK.md 규칙으로 올려 모든 세션이 함께 쓴다.

모델이 바뀌어도 규칙은 파일에 그대로 남는다

세션은 한 번의 실행에서 이어지는 대화 단위다

세션(session) = 터미널에서 claude를 실행해 대화하는 한 묶음

세션이 끝나면 그 대화는 사라진다. 다음 세션은 파일만 읽고 시작한다.

세션이 기억하는 것

- 지금 세션 안에서 주고받은 대화
- 실행한 폴더와 그 위 폴더의 CLAUDE.md
- 세션이 열어 본 파일 내용

그래서 이 구조가 하는 일

- 역할 규칙은 CLAUDE.md에 둔다
- 과제 의도는 INTENT.md에 둔다
- 지적과 교훈은 MEMORY.md에 둔다

CLAUDE.md는 실행 폴더와 상위 폴더에서 함께 읽힌다

CLAUDE.md = 세션이 시작할 때 자동으로 읽는 규칙 파일

my-team/CLAUDE.md

팀 공통 규칙: 모든 세션이 지킨다

my-team/worker-01/CLAUDE.md

역할 규칙: 이 책상만 지킨다

할 일: 배정받은 과제를 구현한다

하지 않을 일: 실서비스 배포는 하지 않는다

→

worker-01 폴더에서 claude를 실행하면

두 파일을 함께 읽고
그 역할의 세션이 된다

위 = 공통, 아래 = 역할

같은 일을 여러 세션이 반복할 때 이 구조가 값을 한다

세션 한두 개를 가끔 쓰는 정도라면 이렇게까지 할 필요는 없다

이 구조가 맞는 경우

- 같은 종류의 과제를 계속 여러 세션에 맡긴다
- 결과물의 품질 기준을 매번 똑같이 지켜야 한다
- 한 세션이 배운 것을 다른 세션도 바로 써야 한다
- 세션 하나하나를 사람이 통제하고 싶다

필요 없는 경우

- 세션 한두 개로 가끔 질문하고 끝낸다
- 한 번 만들고 다시 고치지 않는 일회성 작업이다
- 중간 승인 없이 빨리 끝내는 것이 더 중요하다

01

PART 1 · 팀 구조

역할 폴더와 규칙 파일

폴더 구조를 만들고 역할마다 CLAUDE.md를 쓴다

팀 루트 아래에 역할마다 폴더를 하나씩 둔다

예시 팀 이름은 my-team이다. 이름은 자유롭게 바꿔도 된다

```
my-team/
├── CLAUDE.md
├── HANDBOOK.md
├── MEMORY.md
├── QUALITY_GATES.md
├── leader/ reviewer/ designer/ docs/
├── worker-01/ worker-02/
│   └── tasks/login-redesign/INTENT.md
└── tools/deploy.sh
```

팀 루트

팀 공통 규칙

정식 규칙 모음

지적 사항 메모 (120줄 이내)

검수 관문 다섯 칸

역할 폴더 = 책상 하나

실제로 일하는 워커

과제 하나 = 폴더 하나

테스트 서버만 배포하는 도구

역할은 다섯 가지이며 워커만 여러 명이다

터미널 탭 하나를 열어 역할 폴더로 들어가 claude를 실행하면 그 역할의 세션이 된다

역할 폴더	하는 일	하지 않는 일
leader (리더)	과제를 받고 방향을 묻고 워커에게 배정한다	직접 구현하지 않는다
reviewer (리뷰)	관문 표로 결과를 검수한다	코드를 고치지 않는다
designer (디자이너)	화면 설계안을 그린다	기능 코드를 쓰지 않는다
docs (문서)	문서와 사용 설명을 맞춘다	기능을 바꾸지 않는다
worker-01, 02…	배정받은 과제를 설계하고 구현한다	실서비스에 배포하지 않는다

역할 규칙은 할 일과 하지 않을 일 두 칸으로 쓴다

하지 않을 일을 적어야 남의 일을 덜 건드린다

[출력] Claude가 생성 · worker-01/CLAUDE.md

```
# 역할: 워커 (worker-01)

## 할 일
- 인박스로 배정된 과제만 맡는다
- 과제마다 tasks/<과제명>/INTENT.md를 먼저 만든다
- 설계안을 리더에게 보내고 승인을 기다린다
- 구현 후 QUALITY_GATES.md 다섯 칸을 채워 보고한다

## 하지 않을 일
- 승인 전에 코드를 쓰지 않는다
- 다른 워커의 폴더를 고치지 않는다
- 실서비스(production)에 배포하지 않는다
```

작성 요령

- 직접 쓰지 않고 Claude에게 만들게 한다
- 할 일은 동사로 짧게 쓴다
- 하지 않을 일은 실제 사고가 난 곳부터 적는다

팀 공통 규칙은 루트의 CLAUDE.md 한 곳에 모은다

모든 역할 세션이 시작할 때 이 파일을 함께 읽는다

[출력] Claude가 생성 · my-team/CLAUDE.md

```
# 팀 공통 규칙

- 시작할 때 MEMORY.md와 HANDBOOK.md를 읽는다
- 과제 하나에 세션 하나, 과제 폴더 하나를 쓴다
- 설계 승인(OK) 전에는 코드를 쓰지 않는다
- 보고에는 QUALITY_GATES.md 표를 붙인다
- 메시지는 짧게 쓰고 자세한 내용은 파일 경로로 넘긴다
- 사람에게 지적받으면 MEMORY.md 맨 위에 한 줄로 적는다
- 배포는 tools/deploy.sh로만 한다
```

공통과 역할의 구분

- 모든 세션이 지킬 것은 루트에 둔다
- 한 역할만 지킬 것은 역할 폴더에 둔다
- 같은 규칙을 두 곳에 쓰지 않는다

02

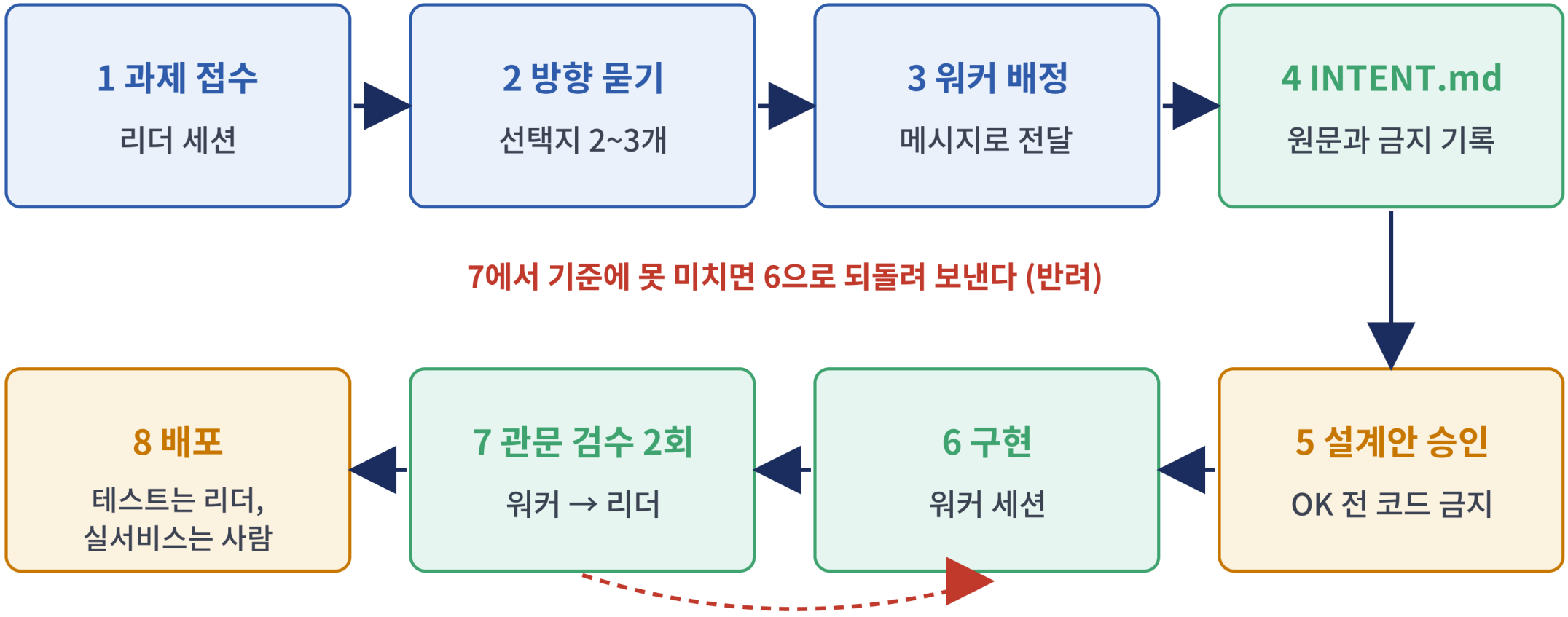
PART 2 · 과제 흐름

과제 하나를 끝까지 따라가기

예시 과제는 로그인 화면 개선이다

과제는 여덟 단계를 거쳐 배포까지 간다

사람이 개입하는 곳은 방향 선택, 설계 승인, 실서비스 배포 세 곳이다



리더는 바로 시작하지 않고 방향부터 묻는다

방향을 두세 가지로 묶어 고를 수 있는 선택지로 보여 준다

[출력] 리더 세션의 응답 예시

```
로그인 화면 개선, 방향을 골라 주세요.

A. 오류 메시지와 입력 검증만 손본다 (작음)
B. 소셜 로그인 버튼을 추가한다 (중간)
C. 휴대폰 화면 기준으로 전체를 다시 배치한다 (큼)

선택하시면 비어 있는 워커에게 배정하겠습니다.
```

왜 먼저 묻는가

- 방향이 틀리면 이후 작업이 모두 버려진다
- 고르는 데 드는 시간은 몇 초다

사용자가 하나를 고르면 리더는 비어 있는 워커에게 메시지로 배정한다

워커는 과제를 받으면 INTENT.md부터 만든다

INTENT.md = 과제의 의도를 고정하는 파일. 과제 폴더마다 하나씩 둔다

[출력] Claude가 생성 · tasks/login-redesign/INTENT.md

```
# 과제: 로그인 화면 개선

## 사용자가 한 말 (원문 그대로)
> 휴대폰에서 버튼이 겹친다. B안으로 가자.

## 하지 말라고 한 것
- 회원가입 화면은 건드리지 않는다

## 결정 기록
- 10/25 방향 B 선택 (사용자)
- 10/25 설계안 v1 승인 (사용자)
```

세 칸을 두는 이유

- 원문을 남겨야 의도가 바뀌지 않는다
- 금지 사항을 남겨야 범위를 넘지 않는다
- 결정을 남겨야 다음 세션이 이어받는다

설계안이 승인되기 전에는 코드를 쓰지 않는다

설계안 = 무엇을 어떤 순서로 바꿀지 적은 짧은 계획

설계안 작성

워커

바꿀 파일과 순서, 화면 변화를 짧게 적어 리더와 사용자에게 보낸다

사용자 확인

사람

방향이 맞으면 OK, 틀리면 한 줄로 고칠 점을 돌려준다

구현 시작

워커

OK가 난 설계안만 구현한다. 설계에 없는 변경은 다시 묻는다

얻는 것

- 엉뚱한 방향으로 한참 구현한 뒤 버리는 일이 사라지고 토큰도 아낀다

검수는 같은 관문 표로 두 번 한다

워커가 먼저 채우고, 리더가 같은 표로 다시 확인한다

관문	통과 기준 (예시)	결과
테스트	모든 테스트가 통과했는가?	통과 / 반려
린트	코드 검사 경고가 하나도 없는가?	통과 / 반려
화면	휴대폰 화면에서 버튼이 겹치지 않는가?	통과 / 반려
문서	바뀐 사용법을 문서에 반영했는가?	통과 / 반려
보고	INTENT.md의 결정 기록을 갱신했는가?	통과 / 반려

표는 QUALITY_GATES.md에 두고 보고할 때마다 그대로 붙인다

반려는 한 줄로, 배포는 역할에 따라 나눈다

하나라도 기준에 못 미치면 바로 되돌려 보낸다

[출력] 리더의 반려 메시지 예시 · 어디서, 무엇이, 어떤 기준에 걸렸는가

```
반려: 로그인 화면 / 휴대폰 폭 360px에서 로그인 버튼과 소셜 버튼이 겹침 / 화면 관문 기준 미달
```

테스트 서버 배포

- 두 번의 검수를 통과하면 리더가 한다
- 도구는 tools/deploy.sh 하나뿐이다

실서비스 배포

- 사람이 직접 한다
- 되돌리기 어려운 일은 세션에 맡기지 않는다

03

PART 3 · 권한 · 통신 · 기억

사고를 막고 배운 것을 남기는 장치

도구로 권한을 막고, 메시지는 짧게, 기억은 파일로 남긴다

되돌리기 어려운 명령은 도구 쪽에서 먼저 막는다

CLAUDE.md에 하지 말라고 써두는 것만으로는 부족할 때가 있다

[출력] Claude가 생성 · tools/deploy.sh

```
#!/bin/bash
# 테스트 서버(staging) 전용 배포 도구
TARGET=$1
if [ "$TARGET" != "staging" ]; then
  echo "이 도구는 staging만 배포합니다."
  echo "실서비스 배포는 사람이 직접 합니다."
  exit 1
fi
echo "staging 배포를 시작합니다..."
# 실제 배포 명령은 이 아래에 둔다
```

원리

- 규칙은 어길 수 있지만 없는 선택지는 고를 수 없다
- 실수로 production을 넣어도 도구가 거절한다
- 데이터 삭제 같은 명령도 같은 방식으로 막는다

세션끼리는 짧은 메시지를 주고받고 자세한 내용은 파일로 넘긴다

받는 세션이 꺼져 있으면 그 역할 폴더의 inbox/에 파일로 남긴다

메시지 전달 방법

- Claude Code의 세션 간 메시지로 보낸다
- 상대가 꺼져 있으면 inbox/에 파일로 둔다

토큰이 걱정되지 않는 이유

- 메시지를 받을 때마다 대화 전체를 다시 읽는다
- 앞부분은 캐시에서 읽어 일반 입력보다 훨씬 싸다

[출력] 리더가 워커에게 보내는 메시지 예시

```
새 과제: 로그인 화면 개선, 방향 B. 자세한 내용은 leader/inbox/login-redesign.md 참고.
INTENT.md부터 만들고 설계안을 보내 주세요.
```

기억은 세 층의 파일로 남긴다

세션은 끝나면 대화를 잊는다. 그래서 다음 세션이 읽을 파일에 적는다



MEMORY.md는 120줄 안으로 유지한다

MEMORY.md는 모든 세션이 시작할 때마다 읽으므로 길어지면 그만큼 비용이 든다

지적이 나온다

1회

지적받은 세션이 MEMORY.md 맨 위에 한 줄로 적는다

같은 지적이 또 나온다

2회

리더가 그 내용을 HANDBOOK.md 정식 규칙으로 올린다

120줄을 넘는다

정리

겹치는 줄은 합치고 오래된 줄은 HANDBOOK.md로 옮긴다

[출력] MEMORY.md 맨 위 예시

```
- 10/25 버튼 간격은 휴대폰 폭 360px에서 먼저 확인한다 (화면 관문 반려)
- 10/24 보고에 관문 표를 빠뜨리지 않는다
```

04

PART 4 · 따라 하기

내 컴퓨터에 팀 만들기

모든 작업은 Claude Code에게 시키고, 사람은 붙여넣고 확인만 한다

팀 폴더와 규칙 파일은 프롬프트 하나로 만든다

빈 작업 폴더에서 claude를 실행하고 아래 프롬프트를 붙여넣는다

[입력] 프롬프트 · Claude Code

```
현재 위치에 my-team 폴더를 만들고 Claude Code 팀 구조를 세팅해 줘.
1) 루트에 CLAUDE.md(팀 공통 규칙), HANDBOOK.md, MEMORY.md, QUALITY_GATES.md를 만든다.
   QUALITY_GATES.md에는 테스트·린트·화면·문서·보고 다섯 칸 표를 넣는다.
2) leader, reviewer, designer, docs, worker-01, worker-02 폴더를 만들고
   폴더마다 CLAUDE.md를 '할 일'과 '하지 않을 일' 두 칸으로 쓴다. 각 폴더에 inbox/도 만든다.
3) 워커 폴더에 tasks/ 를 만들고, 루트 templates/INTENT.md 양식
   (사용자 원문 / 하지 말라고 한 것 / 결정 기록)을 만든다.
4) tools/deploy.sh는 staging만 허용하고 다른 값이면 거절하게 만든다.
5) 공통 규칙: 설계 승인 전 코드 금지, 지적은 MEMORY.md 맨 위에 한 줄,
   MEMORY.md는 120줄 이내, 메시지는 짧게 쓰고 자세한 내용은 파일 경로로 넘긴다.
다 만들면 폴더 트리를 보여 주고 각 CLAUDE.md 내용을 요약해 줘.
```

터미널 탭마다 역할 폴더에서 claude를 실행한다

탭 하나가 책상 하나다. 처음에는 리더와 워커 하나로 시작해도 된다

[입력] 명령어 · 터미널 탭 1 (리더)

```
cd my-team/leader && claude
```

[입력] 명령어 · 터미널 탭 2 (워커)

```
cd my-team/worker-01 && claude
```

[입력] 프롬프트 · 각 세션의 첫 입력

```
네 역할 규칙과 팀 공통 규칙, MEMORY.md, HANDBOOK.md를 읽고
네 역할을 세 줄로 요약해 줘. inbox/에 쌓인 메시지가 있으면 알려 줘.
```

확인할 것

- 세션이 말하는 역할이 그 폴더의 CLAUDE.md와 같으면 제대로 실행된 것이다

첫 과제는 리더 세션에 맡긴다

리더가 선택지를 보여 주면 고르고, 워커의 설계안을 승인한다

[입력] 프롬프트 · 리더 세션

```
과제: 로그인 화면을 개선해 줘.
바로 시작하지 말고 방향을
2~3가지 선택지로 먼저 보여 줘.
내가 고르면 비어 있는 워커에게
inbox 메시지로 배정해 줘.
```

[입력] 프롬프트 · 워커 세션

```
inbox/를 확인하고 새 과제를 맡아 줘.
INTENT.md를 먼저 만들고
설계안을 보여 줘.
내가 OK 하기 전에는 코드를 쓰지 마.
```

[입력] 프롬프트 · 워커 세션 · 구현 후

```
OK. 구현하고 QUALITY_GATES.md 다섯 칸을 채워서 리더에게 보고해 줘.
```

지적은 바로 기억으로 바꾼다

같은 말을 두 번 하지 않게 만드는 습관이다

[입력] 프롬프트 · 지적받은 세션

```
방금 지적한 내용을 MEMORY.md 맨 위에 날짜와 함께 한 줄로 적어 줘.
```

[입력] 프롬프트 · 리더 세션 · 주기적으로

```
MEMORY.md에서 두 번 이상 나온 지적은 HANDBOOK.md 규칙으로 올리고,
120줄을 넘으면 겹치는 줄은 합치고 오래된 줄은 HANDBOOK.md로 옮겨 줘.
```

규칙이 늘어나는 것은 괜찮다

MEMORY.md 목차만 짧게 유지하고 중간중간 정리하면 된다

05

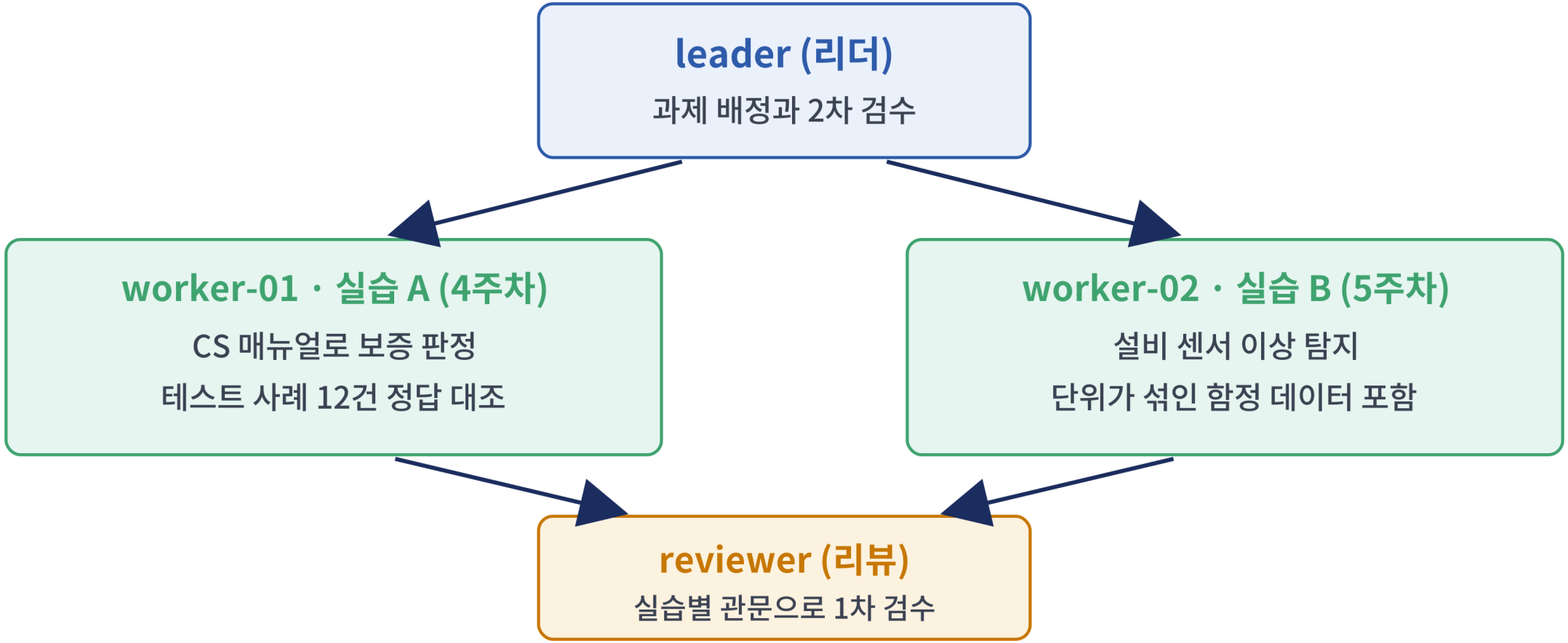
PART 5 · 실무 실습

자동차 3-1 4·5주차 데이터로 팀 돌리기

4주차 보증 판정과 5주차 설비 이상 탐지를 두 워커에게 동시에 맡긴다

두 과제를 두 워커에게 나눠 맡기고 리뷰와 리더가 검수한다

데이터는 3-1 과정 저장소(keerhee/kma-auto-genai-3-1)의 실습용 가상 데이터다



실습마다 정답이나 함정이 심어진 데이터를 쓴다

정답과 함정이 있어야 관문 검수가 실제로 작동하는지 확인할 수 있다

실습	쓰는 파일 (data/ 폴더)	심어진 것
A 보증 판정 (4주차)	Day4_한빛모터스_CS매뉴얼_가상.md Day4_보증판정_테스트사례12_가상.csv	12건마다 정답 판정과 근거 장이 있다
B 설비 이상 탐지 (5주차)	Day5_설비센서_단위혼재_함정_가상.csv Day5_공정별_파일럿평가_가정값_가상.csv	일부 기록장치가 다른 단위로 적혀 있다

준비 원칙

- 파일을 내려받고 폴더에 넣는 일도 모두 Claude Code에게 시킨다

데이터와 실습용 관문은 루트 세션 하나로 준비한다

my-team 폴더에서 claude를 실행하고 붙여넣는다

[입력] 프롬프트 · Claude Code · my-team 루트

```
github.com/keerhee/kma-auto-genai-3-1 저장소에서 data/Day4, data/Day5 폴더만
my-team/data/ 아래로 내려받아 줘. 내려받은 파일 목록과 각 파일의 행 수를 보여 줘.

그다음 실습용 관문 파일 두 개를 만들어 줘.
1) gates/warranty.md (실습 A 보증 판정)
   다섯 칸은 정답 일치, 근거 장 표기, 판정 중단 연결, 추측 금지, 보고로 한다.
2) gates/sensor.md (실습 B 설비 이상 탐지)
   다섯 칸은 단위 확인, 이상 근거, 단발과 지속 구분, 투자 계산 재현, 보고로 한다.
칸마다 '통과 기준'을 질문 한 문장으로 적고, 결과 칸은 통과/반려로 둔다.

마지막으로 leader, reviewer의 CLAUDE.md에
'실습 A는 gates/warranty.md, 실습 B는 gates/sensor.md로 검수한다'는 줄을 추가해 줘.
```

리더에게 과제를 넘기면 방향 선택지부터 돌아온다

실습 A = CS 매뉴얼을 근거로 고객 문의의 보증 여부를 판정하는 과제

[입력] 프롬프트 · leader 세션

```
새 과제(실습 A):
data/Day4 CS 매뉴얼을 근거로
보증 문의를 판정하는 기능을 만든다.
테스트는 보증판정 테스트사례 12건으로
하고 gates/warranty.md로 검수한다.

바로 배정하지 말고 방향을 2~3개
선택지로 먼저 보여 줘.
내가 고르면 worker-01 inbox에
짧은 메시지로 배정해 줘.
```

[출력] 리더의 선택지 예시

```
A. 매뉴얼 1장 규칙만 판정표로 옮긴다
   (작음, 설명 문장 없음)

B. 매뉴얼 검색 + 규칙 판정 + 판정 근거
   설명을 붙인 판정기를 만든다 (중간)

C. B에 상담원용 화면까지 붙인다 (큼)

→ B를 고르면 worker-01에 배정합니다.
```

워커는 금지 사항을 INTENT.md에 먼저 고정한다

매뉴얼에 없는 내용을 추측하지 않는다는 원칙을 처음부터 적어 둔다

[입력] 프롬프트 · worker-01 세션

```
inbox를 확인하고 실습 A를 맡아 줘.
tasks/warranty-judge/INTENT.md를
템플릿대로 만들고, 하지 말라고 한 것에
아래 두 가지를 넣어 줘.
- 매뉴얼에 없는 내용은 추측하지 않는다
- 안전·법적 조치 언급은 판정하지 않고
  매뉴얼 8장 기준으로 담당자에게 넘긴다

그다음 설계안만 보여 줘.
내가 OK 하기 전에는 코드를 쓰지 마.
```

[출력] 설계안 예시

```
1) 매뉴얼 1장 표를 판정 규칙으로 옮긴다
   (일반 3년/6만km, EV 8년/16만km,
    영업용 일반 부품 2년/4만km ...)
2) 보증 시작일 = 최초등록일로 고정
3) 판정 결과는 네 가지 중 하나
   보증 적용 / 보증 제외 /
   추가 확인 / 판정 중단
4) 결과마다 근거 장(1장, 8장-1 ...)을 단다
5) 12건을 돌려 정답 열과 비교한다
```

보증 판정 관문은 정답 대조와 안전 연결을 함께 본다

gates/warranty.md에 들어가는 다섯 칸이다

관문	통과 기준	이 데이터에서 걸리기 쉬운 곳
정답 일치	12건 판정이 정답 열과 모두 같은가?	T04 영업용은 2년/4만km로 줄어든다
근거 장 표기	판정마다 매뉴얼 장 번호를 달았는가?	T08은 8장-1, T12는 8장-2다
판정 중단 연결	안전·법적 언급을 담당자에게 넘겼는가?	T08 고전압 경고등, T12 소송 언급
추측 금지	근거가 부족하면 추가 확인으로 두었는가?	T07 최초등록일 없음, T11 앱 수치
보고	INTENT.md 결정 기록을 갱신했는가?	방향 B 선택과 설계 승인 날짜

리뷰는 걸린 사례를 한 줄로 반려하고 워커는 그 줄만 고친다

가장 흔한 실수는 출고일을 보증 시작일로 쓰는 것이다

[입력] 프롬프트 · reviewer 세션

```
worker-01의 실습 A 보고를 gates/warranty.md 다섯 칸으로 검수해 줘.
미달이면 '반려: 어디서 / 무엇이 / 어떤 기준' 한 줄로 써서 worker-01 inbox에 넣어 줘.
```

[출력] 리뷰의 반려 예시

```
반려: T07 / 출고일 2024-01-05를 시작일로 보고 '보증 적용' 판정 / 보증 시작일은 최초등록일(1장) → 추가 확인이 정답
반려: T11 / 앱 잔존용량 68%로 '보증 적용' 판정 / 용량은 지정 서비스센터 진단으로만 판정(1장) → 추가 확인
```

[입력] 프롬프트 · worker-01 세션

```
inbox의 반려 두 줄만 고쳐 줘. 다른 사례 판정은 바꾸지 마.
고친 뒤 12건을 다시 돌려 정답 일치 수를 보여 주고, 지적 내용을 MEMORY.md 맨 위에 한 줄로 적어 줘.
```

두 번째 과제는 다른 워커에게 동시에 배정한다

실습 B = 프레스 모터와 용접로봇 감속기 센서에서 이상 징후를 찾는 과제

[입력] 프롬프트 · leader 세션

```
새 과제(실습 B):
data/Day5 설비센서 단위혼재 파일에서
설비 A·B의 이상 징후를 찾고, 파일럿 평가
가정값으로 예지보전을 먼저 붙일 공정을
추천한다. 검수는 gates/sensor.md로 한다.

방향 선택지를 먼저 보여 주고,
내가 고르면 worker-02 inbox에 배정해 줘.
실습 A 진행 상황도 한 줄로 알려 줘.
```

[출력] 리더의 선택지 예시

```
A. 기준선 대비 이상 시점만 표로 뽑는다

B. 단위 정리 → 이상 시점 → 단발/지속
   구분 → 공정 추천까지 한 보고서로

C. B에 실시간 모니터링 화면을 붙인다

실습 A: worker-01 반려 2건 수정 중
```

분석보다 단위 확인을 먼저 하게 시킨다

기록장치가 바뀌면 같은 열에 다른 단위가 섞일 수 있다

[입력] 프롬프트 · worker-02 세션

```
inbox를 확인하고 실습 B를 맡아 줘. INTENT.md를 만들고 설계안 승인을 받은 뒤 진행해 줘.

분석 전에 '기록장치'별로 진동·온도·압력의 평균과 범위를 표로 보여 주고,
다른 장치와 크게 다른 값이 있으면 단위 차이인지 먼저 판단해 줘.
단위를 바꿀 때는 어떤 환산식을 썼는지 INTENT.md 결정 기록에 남겨 줘.
단위를 확인하기 전에는 이상 판정을 하지 마.
```

왜 이 순서인가

- 단위를 확인하지 않으면 정상 설비가 과열·고압으로 보이는 오탐이 생긴다

설비 이상 관문은 단위와 이상의 성격을 따로 본다

gates/sensor.md에 들어가는 다섯 칸이다

관문	통과 기준	이 데이터에서 걸리기 쉬운 곳
단위 확인	기록장치별 단위를 맞춘 뒤 분석했는가?	로거2 온도는 °F, 로거4 진동은 in/s, 압력은 psi
이상 근거	정상 구간과 비교한 수치를 제시했는가?	설비 B 진동 1.0 → 2.9 mm/s
단발과 지속	한 번 튄 값과 계속 오르는 값을 구분했는가?	설비 A 12:40 진동 3.5는 단발
투자 계산 재현	연간정지손실÷초기투자를 다시 계산했는가?	가정값이므로 가정이라고 밝힌다
보고	INTENT.md에 환산식과 결정을 남겼는가?	°F→°C, in/s→mm/s, psi→bar

단위 함정은 반려 한 줄로 바로 드러난다

단위를 확인하라고 시켜도 놓치는 경우를 대비한 두 번째 검수다

[출력] 리뷰의 반려 예시

```
반려: 설비 A 10:00~10:40 / 온도 124°C로 '과열' 판정 / 로거2(교체품)는 °F 기록, 약 51°C로 정상
반려: 설비 B 13:20~13:40 / 진동 0.043으로 '센서 이상' 판정 / 로거4는 in/s 기록, 약 1.1 mm/s로 정상
반려: 설비 B 15:00 / 압력 1690으로 '고압' 판정 / 로거4는 psi 기록, 약 116.5 bar로 정상
```

[입력] 프롬프트 · worker-02 세션

```
반려 세 줄을 반영해 단위를 환산하고 이상 판정을 다시 해 줘. 환산식은 INTENT.md에 남기고,
같은 실수를 막도록 MEMORY.md 맨 위에 '분석 전 기록장치별 단위 확인' 한 줄을 적어 줘.
```

함정이 하는 일: 관문이 실제로 걸러 내는지 확인하는 장치다

통과한 보고서는 지속 상승 설비를 먼저 지목한다

단위를 맞추고 나면 진짜 이상은 설비 B 한 곳에 남는다

[출력] worker-02의 보고 예시 (관문 5칸 통과)

1. 설비 B (용접로봇 감속기) — 점검 우선
   온도 13:00 이후 55℃ → 65℃로 계속 상승
   진동 15:00부터 1.3 → 2.9 mm/s로 계속 상승
   → 단발이 아니라 진행 중인 열화로 본다

2. 설비 A (프레스 메인모터) — 관찰
   12:40 진동 3.5 mm/s 한 번 튄 뒤 바로 정상
   → 단발 충격, 반복되면 재검토

3. 예지보전 파일럿 추천: 프레스 공정
   연간정지손실÷초기투자 2.59배로 가장 큼
   (가정값 기준, 패널 재고 버퍼로 실제는 더 작을 수 있음)

확인 포인트

- 단발과 지속을 나눠 판정했는가
- 투자 수치가 가정값임을 밝혔는가
- 실제 정비 지시는 사람이 내린다

두 실습의 지적은 HANDBOOK 규칙으로 올라간다

같은 종류의 실수가 두 과제에서 반복되면 팀 규칙이 된다

[출력] MEMORY.md 맨 위

```
- 보증 시작일은 최초등록일, 출고일 아님 (실습 A)
- 앱 수치로 배터리 용량 판정 금지 (실습 A)
- 분석 전 기록장치별 단위 확인 (실습 B)
```

[입력] 프롬프트 · leader 세션

```
MEMORY.md에서 두 실습에 공통인
'근거가 확인되지 않은 수치로 판정하지
않는다'를 HANDBOOK.md 규칙으로 올리고,
합쳐진 줄은 MEMORY.md에서 지워 줘.
```

[출력] HANDBOOK.md에 추가된 규칙

```
## 근거 확인 규칙 (실습 A·B 반려에서 승격)
- 판정에 쓰는 날짜·수치·단위는 정해진 출처로 확인한 것만 쓴다.
- 확인되지 않으면 판정하지 않고 '추가 확인'으로 두거나 환산 근거를 먼저 남긴다.
```

06

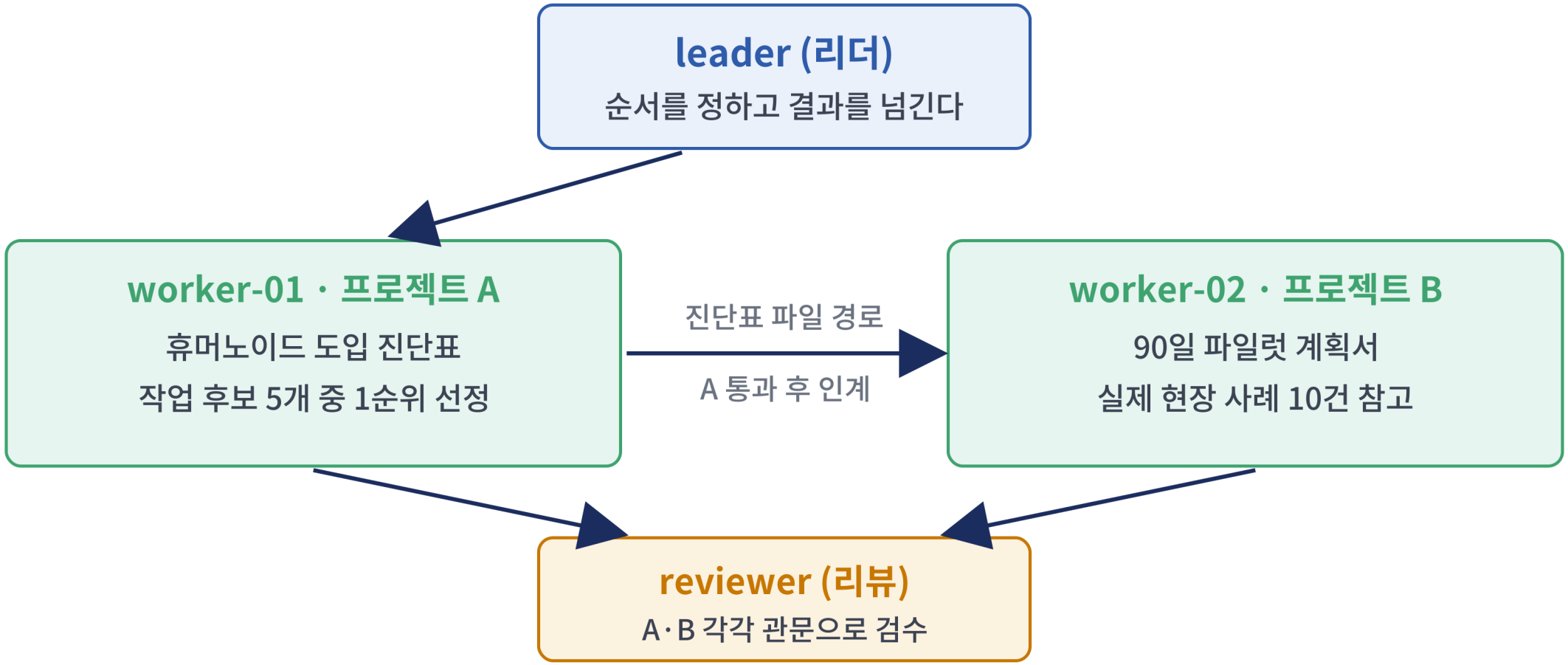
PART 6 · 로봇 특강 실습

휴머노이드 도입 진단과 90일 파일럿

앞 과제의 결과를 다음 워커에게 넘기는 순차 인계를 연습한다

프로젝트 A가 통과해야 프로젝트 B가 시작된다

Part 5는 두 과제를 동시에 돌렸고, Part 6은 앞 결과를 다음 과제로 넘긴다



데이터 두 개와 관문 두 개로 실습을 준비한다

데이터는 3-1 과정 저장소 data/Robot_Special 폴더에 있다

파일	내용	주의할 점
Robot_작업후보5_평가매트릭스_템플릿.csv	조립 공정 작업 후보 5개와 평가 기준 5개	기준 칸이 비어 있다
Robot_휴머노이드·물류로봇_현장파일럿사례_실데이터.csv	Figure·Atlas·Spot·Stretch·Digit·Apollo 현장 사례 10건	KPI가 '확인 불가'인 사례가 많다

[입력] 프롬프트 · Claude Code · my-team 루트

```
github.com/keerhee/kma-auto-genai-3-1 저장소의 data/Robot_Special 폴더를 my-team/data/ 아래로 내려받아 줘.
그다음 gates/robot_diagnosis.md와 gates/robot_pilot.md를 다섯 칸 관문 표로 만들어 줘.
진단 관문은 기준 완비, 대안 비교, 근거 문장, TCO와 안전, 보고로 하고,
파일럿 관문은 측정 가능, 중단 기준, 출처 일치, 안전과 사람 역할, 보고로 해 줘. 통과 기준은 질문 한 문장으로 적어 줘.
```

리더는 두 과제의 순서부터 정해 묻는다

프로젝트 A = 조립 공정 작업 5개 가운데 휴머노이드 1순위 작업을 고르는 진단

[입력] 프롬프트 · leader 세션

```
로봇 특강 실습을 진행한다.
A: 휴머노이드 도입 진단표
   (평가 매트릭스 템플릿 사용)
B: A의 1순위 작업으로 90일 파일럿
   계획서 (현장 사례 CSV 참고)

B는 A가 관문을 통과한 뒤에 시작해.
진행 순서와 배정안을 선택지로
먼저 보여 주고, 내가 고르면
A부터 worker-01에 배정해 줘.
```

[출력] 리더의 선택지 예시

```
1. A → B 순차 (권장)
   A 통과 후 진단표 경로를 worker-02에
   넘긴다

2. A·B 동시 시작
   B는 '부품 정렬'을 1순위로 가정하고
   A 결과에 맞춰 고친다

가정이 틀리면 B를 다시 써야 하므로
1번을 권합니다.
```

워커는 '가장 힘든 일부터'를 금지 사항으로 적는다

특강의 도입 공식은 검증된 단순 작업부터 시작하는 것이다

[입력] 프롬프트 · worker-01 세션

```
inbox를 확인하고 프로젝트 A를 맡아 줘. INTENT.md의 하지 말라고 한 것에 다음을 적어 줘.
- 가장 힘든 작업을 1순위로 두지 않는다
- AMR·비전 AI·코봇이 더 맞는 작업을 휴머노이드에 배정하지 않는다
- TCO 수치는 근거 범위만 쓰고 '추정'으로 표시한다
템플릿의 다섯 기준(반복성, 위험·중량, 정형성, 협업 필요도, 실패 시 영향)을 모두 채우고 칸마다 근거를 한 줄 달아 줘.
```

[출력] 진단표 예시 · worker-01/tasks/robot-diagnosis/diagnosis.csv

작업 후보	반복성	위험·중량	정형성	협업 필요	실패 영향	대안 로봇 검토	종합
부품 정렬(시퀀싱)	상	중	상	하	중	휴머노이드 적합	1순위
중량물 운반	상	상	상	하	중	코봇·리프트와 비교	2순위
나사 체결 보조	상	하	중	중	상	코봇이 이미 가능	3순위
외관 검사 보조	상	하	중	상	상	비전 AI가 먼저	제외
공정 간 부품 이동	상	중	상	하	하	AMR이 이미 적임	제외

진단 관문은 대안 로봇과 비교했는지를 따로 본다

gates/robot_diagnosis.md에 들어가는 다섯 칸이다

관문	통과 기준	이 데이터에서 걸리기 쉬운 곳
기준 완비	다섯 기준을 모든 후보에 채웠는가?	실패 시 영향 열을 빠뜨린다
대안 비교	다른 로봇이 더 맞는 작업을 걸렀는가?	외관 검사, 공정 간 이동
근거 문장	칸마다 현장 근거를 한 줄 달았는가?	'효율적이다' 같은 빈말
TCO·안전	숨은 비용과 정지 조건을 적었는가?	수치에 '추정' 표기 누락
보고	INTENT.md 결정 기록을 갱신했는가?	1순위 선정 이유 두 가지

반려는 대안 로봇을 놓친 칸을 정확히 짚는다

흔한 실수는 휴머노이드를 만능으로 보는 것이다

[출력] 리뷰의 반려 예시

```
반려: 외관 검사 보조 / 휴머노이드 2순위로 배정 / 대안 비교 기준 미달 — 비전 AI가 먼저 검토 대상
반려: 진단표 전체 / '실패 시 영향' 열이 비어 있음 / 기준 완비 미달 — 다섯 기준 모두 필요
반려: TCO 문단 / '대당 2억 원'을 근거 없이 단정 / TCO·안전 미달 — 범위와 '추정' 표기 필요
```

[입력] 프롬프트 · worker-01 세션

```
반려 세 줄만 고쳐 줘. 고친 뒤 '이 진단에서 다른 로봇이 더 적합한 작업을 휴머노이드에 배정한 곳을 찾아줘'로
스스로 역검토하고, 결과를 보고에 붙여 줘. 지적 내용은 MEMORY.md 맨 위에 한 줄로 적어 줘.
```

역검토 프롬프트는 관문 앞에 두는 자체 점검이다

리더는 통과한 진단표의 경로만 다음 워커에게 넘긴다

프로젝트 B = A의 1순위 작업을 로봇 임무로 정의하고 90일 파일럿을 설계하는 과제

[입력] 프롬프트 · leader 세션

```
프로젝트 A가 두 관문을 통과했으면
worker-02 inbox에 프로젝트 B를 배정해.
메시지에는 진단표 파일 경로와
1순위 작업 이름만 적고,
자세한 내용은 파일을 읽게 해.
```

[출력] worker-02 inbox 메시지

```
새 과제: 프로젝트 B (90일 파일럿)
1순위 작업: 부품 정렬(시퀀싱)
근거: worker-01/tasks/robot-diagnosis/
      diagnosis.csv, INTENT.md
참고: data/Robot_Special 현장 사례 CSV
```

짧은 메시지와 파일 경로

- 메시지가 짧을수록 캐시가 잘 쓰이고, 파일을 읽게 하면 내용이 바뀌지 않는다

워커에게 '확인 불가'를 지어내지 말라고 먼저 못 박는다

현장 사례 CSV 10건 중 다수는 공개된 KPI가 없다

[입력] 프롬프트 · worker-02 세션

```
inbox를 확인하고 프로젝트 B를 맡아 줘. 진단표와 INTENT.md를 먼저 읽어 줘.
1) 부품 정렬을 로봇 임무로 정의해 줘: 입력, 성공 기준, 예외 5개, 사람이 개입하는 지점, 안전 규칙.
2) 현장 사례 CSV에서 기간·KPI·확대 조건을 출처 URL과 함께 표로 뽑아 줘.
   '확인 불가'로 적힌 칸은 그대로 두고 숫자를 지어내지 마. 요구조건과 달성 성과는 구분해 줘.
3) 이를 참고해 90일 3단계 계획을 짜고 단계마다 범위, KPI 3개, 다음 단계 조건, 중단 기준을 넣어 줘.
```

[출력] 파일럿 계획 예시 (수치는 모두 추정)

단계	범위	핵심 KPI	다음 단계 조건
시범 1~30일	1개 셀, 주간 근무만	정렬 오류율, 사람 개입 횟수	안전사고 0건, 개입 횟수 감소
1교대 31~60일	라인 1교대 전체	사이클타임이 택트 이내인가	KPI 2주 연속 달성
2교대 61~90일	2교대, 야간 포함	가동률, TCO 추정 대비 실적	확대 승인 또는 중단 보고

중단 기준: 안전 이상 1건, 또는 사람 개입이 2주 연속 기준을 넘으면 멈춘다

파일럿 관문은 KPI를 잴 수 있는지와 출처가 맞는지를 본다

gates/robot_pilot.md에 들어가는 다섯 칸이다

관문	통과 기준	이 데이터에서 걸리기 쉬운 곳
측정 가능	KPI가 모두 숫자로 잴 수 있는가?	'생산성 향상' 같은 KPI
중단 기준	확대 조건과 함께 멈출 조건이 있는가?	확대 조건만 적고 끝낸다
출처 일치	사례 수치가 CSV와 출처 그대로인가?	Atlas·Figure 03은 KPI 확인 불가
안전·사람	정지 조건과 사람의 역할을 적었는가?	감독·복구 담당자 누락
보고	INTENT.md 결정 기록을 갱신했는가?	RaaS로 시작할지에 대한 결정

사례 수치를 지어내거나 요구조건을 성과로 바꾸면 반려된다

공개 사례를 인용할 때 가장 많이 걸리는 두 가지 함정이다

[출력] 리뷰의 반려 예시

```
반려: 사례 표 Atlas 행 / '정렬 정확도 98%' 인용 / 출처 일치 미달 — CSV에는 KPI '확인 불가'
반려: 사례 표 Figure 02 행 / '사이클 84초 달성' / 출처 일치 미달 — 84초는 BMW 요구조건, 성과 아님
반려: 2교대 단계 / KPI '생산성 향상' / 측정 가능 미달 — 숫자로 잴 수 있는 지표로 바꿔야 함
```

[입력] 프롬프트 · worker-02 세션

```
반려 세 줄만 고쳐 줘. 고친 뒤 '이 계획이 실패할 경로 3개를 찾아줘'로 역검토하고 보고에 붙여 줘.
'공개 사례 수치는 CSV 그대로, 요구조건과 성과는 구분'을 MEMORY.md 맨 위에 한 줄로 적어 줘.
```

실제 공개 성과의 예: Figure 02는 11개월간 부품 9만 개 이상을 적재했다

세 실습에서 쌓인 지적은 하나의 근거 규칙으로 모인다

Part 5의 보증·센서 실습과 Part 6의 로봇 실습이 같은 실수를 공유한다

[출력] MEMORY.md 맨 위 (로봇 실습)

- 대안 로봇과 비교한 뒤 휴머노이드 배정
- TCO 수치는 범위와 '추정' 표기
- 공개 사례 수치는 CSV 그대로 인용
- 요구조건과 달성 성과를 구분

[입력] 프롬프트 · leader 세션

```
MEMORY.md의 로봇 실습 지적 가운데
HANDBOOK.md '근거 확인 규칙'과 같은
취지는 그 규칙에 합치고,
합쳐진 줄은 MEMORY.md에서 지워 줘.
```

[출력] HANDBOOK.md 근거 확인 규칙 (보강)

- 판정에 쓰는 날짜·수치·단위는 정해진 출처로 확인한 것만 쓴다.
- 공개 사례의 수치는 출처 그대로 쓰고, 요구조건·목표를 달성 성과로 바꾸지 않는다.
- 확인되지 않으면 '확인 불가' 또는 '추정'으로 표시한다.

역할 이름과 관문 칸만 바꾸면 다른 업무에도 쓸 수 있다

개발 대신 강의 콘텐츠 제작에 옮긴 예시다

구성 요소	개발 팀	강의 콘텐츠 팀
리더	과제를 배정하고 검수한다	강의 구성을 정하고 장별로 배정한다
워커	기능을 구현한다	장별 원고와 덱을 만든다
디자이너	화면을 설계한다	도해와 차트를 그린다
관문 다섯 칸	코드가 돌고 화면이 깨지지 않는가?	사실이 맞고 그림이 잘 읽히는가?
사람이 하는 일	실서비스에 배포한다	최종 강의안을 확정해 공개한다

중간 승인은 느리지만 방향을 잃지 않게 한다

써 보고 확인된 장점과 감수할 점이다

장점

- 누가 무엇을 하는지 폴더만 봐도 보인다
- 같은 말을 두 번 할 일이 줄어든다
- 세션 하나하나를 거의 완벽하게 제어한다

감수할 점

- 큰 작업 중간마다 사람이 끼어들어 느려진다
- 규칙 파일이 계속 늘어나 정리가 필요하다
- 탭과 세션을 여러 개 관리해야 한다

여섯 가지를 갖추면 팀이 돌아간다

처음 세팅할 때 하나씩 확인한다

확인할 것	어디에 있나
역할마다 폴더가 있고 그 안에 CLAUDE.md가 있는가?	my-team/<역할>/CLAUDE.md
공통 규칙에 설계 승인 전 코드 금지가 들어 있는가?	my-team/CLAUDE.md
과제마다 INTENT.md를 먼저 만드는가?	worker-NN/tasks/<과제>/
보고에 관문 다섯 칸 표가 붙는가?	QUALITY_GATES.md
배포 도구가 staging만 허용하는가?	tools/deploy.sh
지적이 MEMORY.md에 쌓이고 120줄을 넘지 않는가?	MEMORY.md, HANDBOOK.md